IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Hierarchical method of clustering- II

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

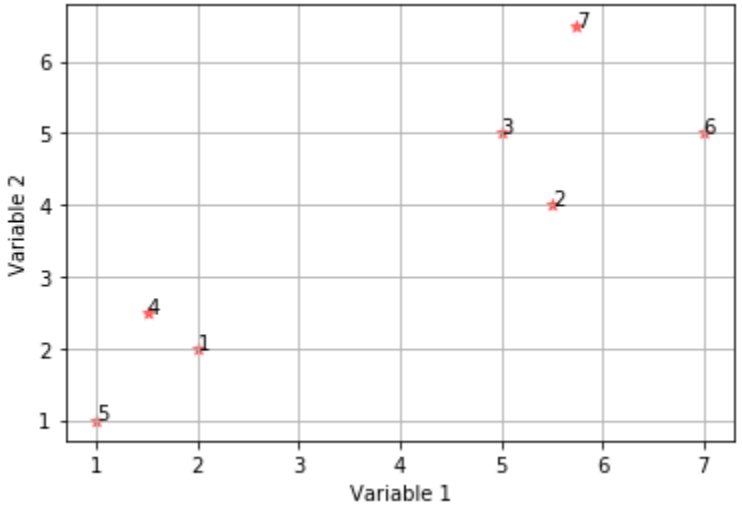
- Agglomerative hierarchical algorithm
- Python demo

Example for Hierarchical Agglomerative Clustering (HAC)

- A data set consisting of seven objects for which two variables were measured.

Object	Variable 1	Variable 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Scatter plot



Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

- Calculate Euclidean Distance and create the distance matrix.

$$\text{Distance}[(x_1, y_1), (x_2, y_2)] = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance (1,2)

$$(2.00, 2.00)\ (5.50, 4.00) = \sqrt{(5.50 - 2.00)^2 + (4.00 - 2.00)^2} = 4.02$$

Distance (1,3)

$$(2.00, 2.00)\ (5.00, 5.00) = \sqrt{(5.00 - 2.00)^2 + (5.00 - 2.00)^2} = 4.24$$

Distance (1,4)

$$(2.00, 2.00)\ (1.50, 2.50) = \sqrt{(1.50 - 2.00)^2 + (2.50 - 2.00)^2} = 0.71$$

Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Distance (1,5)

$$(2.00, 2.00)\ (1.00, 1.00) = \sqrt{(1.00 - 2.00)^2 + (1.00 - 2.00)^2}$$

$$= 1.41$$

Distance (1,6)

$$(2.00, 2.00)\ (7.00, 5.00) = \sqrt{(7.00 - 2.00)^2 + (5.00 - 2.00)^2} = 5.83$$

Distance (1,7)

$$(2.00, 2.00)\ (5.75, 6.50) = \sqrt{(5.75 - 2.00)^2 + (6.50 - 2.00)^2} = 5.86$$

Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Distance (2,3)

$$(5.50, 4.00)\ (5.00, 5.00) = \sqrt{(5.00 - 5.50)^2 + (5.00 - 4.00)^2}$$

$= 1.12$

Distance (2,4)

$$(5.50, 4.00)\ (1.50, 2.50) = \sqrt{(1.50 - 5.50)^2 + (2.50 - 4.00)^2} = 4.27$$

Distance (2,5)

$$(5.50, 4.00)\ (1.00, 1.00) = \sqrt{(1.00 - 5.50)^2 + (1.00 - 4.00)^2} = 5.41$$

Distance (2,6)

$$(5.50, 4.00)\ (7.00, 5.00) = \sqrt{(7.00 - 5.50)^2 + (5.00 - 4.00)^2} = 1.80$$

Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Distance (2,7)

$$(5.50, 4.00)\ (5.75, 6.50) = \sqrt{(5.75 - 5.50)^2 + (6.50 - 4.00)^2}$$

$$= 2.51$$

Distance (3,4)

$$(5.00, 5.00)\ (1.50, 2.50) = \sqrt{(1.50 - 5.00)^2 + (2.50 - 5.00)^2} = 4.30$$

Distance (3,5)

$$(5.00, 5.00)\ (1.00, 1.00) = \sqrt{(1.00 - 5.00)^2 + (1.00 - 5.00)^2} = 5.66$$

Distance (3,6)

$$(5.00, 5.00)\ (7.00, 5.00) = \sqrt{(7.00 - 5.00)^2 + (5.00 - 5.00)^2} = 2.00$$

Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Distance (3,7)

$$(5.00, 5.00)\ (5.75, 6.50) = \sqrt{(5.75 - 5.00)^2 + (6.50 - 5.00)^2}$$

$$= 1.68$$

Distance (4,5)

$$(1.50, 2.50)\ (1.00, 1.00) = \sqrt{(1.00 - 1.50)^2 + (1.00 - 2.50)^2} = 1.58$$

Distance (4,6)

$$(1.50, 2.50)\ (7.00, 5.00) = \sqrt{(7.00 - 1.50)^2 + (5.00 - 2.50)^2} = 6.04$$

Distance (4,7)

$$(1.50, 2.50)\ (5.75, 6.50) = \sqrt{(5.75 - 1.50)^2 + (6.50 - 2.50)^2} = 5.84$$

Example for HAC

Object	Var 1	Var 2
1	2.00	2.00
2	5.50	4.00
3	5.00	5.00
4	1.50	2.50
5	1.00	1.00
6	7.00	5.00
7	5.75	6.50

Distance (5,6)

$$(1.00, 1.00)\ (7.00, 5.00) = \sqrt{(7.00 - 1.00)^2 + (5.00 - 1.00)^2}$$

$$= 7.21$$

Distance (5,7)

$$(1.00, 1.00)\ (5.75, 6.50) = \sqrt{(5.75 - 1.00)^2 + (6.50 - 1.00)^2} = 7.27$$

Distance (6,7)

$$(7.00, 5.00)\ (5.75, 6.50) = \sqrt{(5.75 - 7.00)^2 + (6.50 - 5.00)^2} = 1.95$$

Distance Matrix

- The distance matrix is-

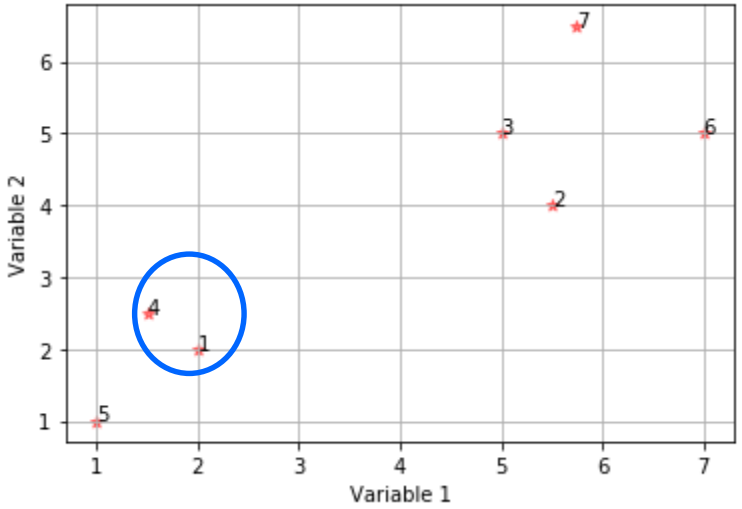
	1	2	3	4	5	6	7
1	0.0						
2	4.0	0.0					
3	4.2	1.1	0.0				
4	0.7	4.3	4.3	0.0			
5	1.4	5.4	5.7	1.6	0.0		
6	5.8	1.8	2.0	6.0	7.2	0.0	
7	5.9	2.5	1.7	5.8	7.3	2.0	0.0

Example for HAC

- Select minimum element to build first cluster formation-

	1	2	3	4	5	6	7
1	0.0						
2	4.0	0.0					
3	4.2	1.1	0.0				
4	0.7	4.3	4.3	0.0			
5	1.4	5.4	5.7	1.6	0.0		
6	5.8	1.8	2.0	6.0	7.2	0.0	
7	5.9	2.5	1.7	5.8	7.3	2.0	0.0

Example for HAC



Example for HAC

- Recalculate distance to update distance matrix

	1	2	3	4	5	6	7
1	0.0						
2	4.0	0.0					
3	4.2	1.1	0.0				
4	0.7	4.3	4.3	0.0			
5	1.4	5.4	5.7	1.6	0.0		
6	5.8	1.8	2.0	6.0	7.2	0.0	
7	5.9	2.5	1.7	5.8	7.3	2.0	0.0

- MIN[dist(1,4), 2] = MIN(dist(1,2), (4,2))
  = MIN(4.0, 4.3) = 4.0
- MIN[dist(1,4), 3] = MIN(dist(1,3), (4,3))
  = MIN(4.2, 4.3) = 4.2
- MIN[dist(1,4), 5] = MIN(dist(1,5), (4,5)) = MIN(1.4, 1.6) = 1.4
- MIN[dist(1,4), 6] = MIN(dist(1,6), (4,6)) = MIN(5.8, 6.0) = 5.8
- MIN[dist(1,4), 7] = MIN(dist(1,7), (4,7)) = MIN(5.9, 5.8) = 5.8

Example for HAC

- Updated distance matrix for the cluster (1, 4)

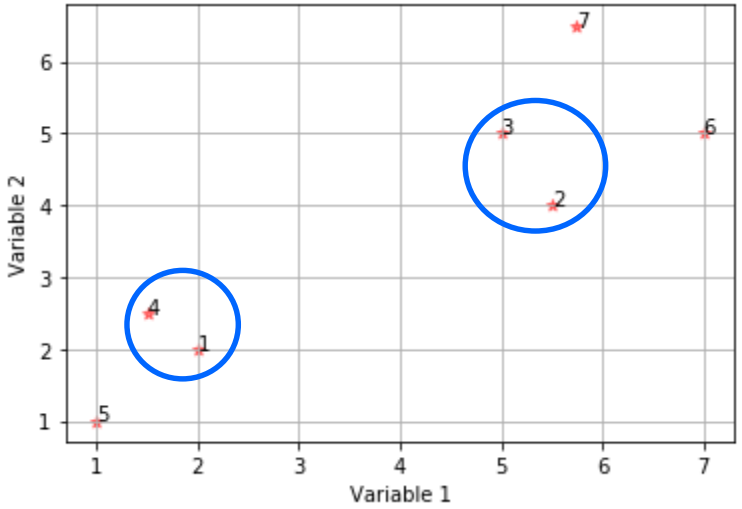
	1,4	2	3	5	6	7
1,4	0.0					
2	4.0	0.0				
3	4.2	1.1	0.0			
5	1.4	5.4	5.7	0.0		
6	5.8	1.8	2.0	7.2	0.0	
7	5.8	2.5	1.7	7.3	2.0	0.0

Example for HAC

- Select minimum element to build next cluster formation-

	1,4	2	3	5	6	7
1,4	0.0					
2	4.0	0.0				
3	4.2	1.1	0.0			
5	1.4	5.4	5.7	0.0		
6	5.8	1.8	2.0	7.2	0.0	
7	5.8	2.5	1.7	7.3	2.0	0.0

Example for HAC



Example for HAC

	1,4	2	3	5	6	7
1,4	0.0					
2	4.0	0.0				
3	4.2	1.1	0.0			
5	1.4	5.4	5.7	0.0		
6	5.8	1.8	2.0	7.2	0.0	
7	5.8	2.5	1.7	7.3	2.0	0.0

- Recalculate distance to update distance matrix

  - MIN[dist(2,3), (1,4)] = MIN(dist(2,(1,4)), (3,(1,4))
    = MIN(4.0, 4.2) = 4.0
  - MIN[dist(2,3), 5] = MIN(dist(2,5), (3,5)) = MIN(5.4, 5.7) = 5.4
  - MIN[dist(2,3), 6] = MIN(dist(2,6), (3,6)) = MIN(1.8, 2.0) = 1.8
  - MIN[dist(2,3), 7] = MIN(dist(2,7), (3,7)) = MIN(2.5, 1.7) = 1.7

Example for HAC

- Updated distance matrix for the cluster (2, 3)

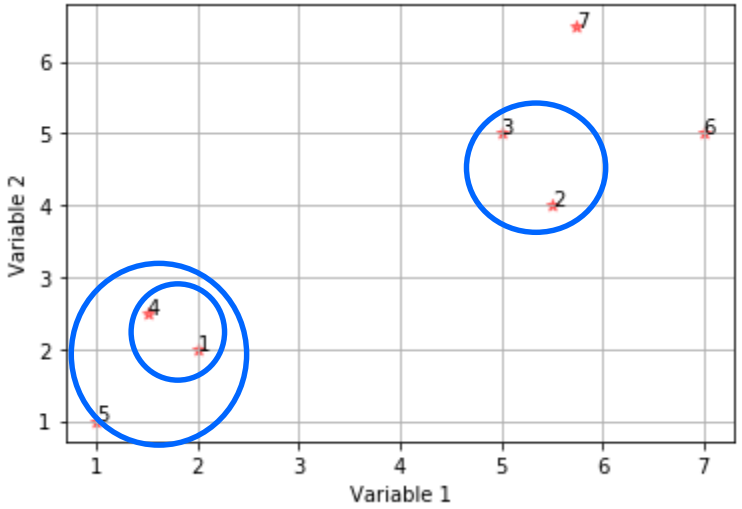
	1,4	**2,3**	**5**	**6**	**7**
1,4	0.0				
2,3	4.0	0.0			
5	1.4	5.4	0.0		
6	5.8	1.8	7.2	0.0	
7	5.8	1.7	7.3	2.0	0.0

Example for HAC

- Select minimum element to build next cluster formation-

	1,4	**2,3**	**5**	**6**	**7**
1,4	0.0				
2,3	4.0	0.0			
5	1.4	5.4	0.0		
6	5.8	1.8	7.2	0.0	
7	5.8	1.7	7.3	2.0	0.0

Example for HAC



Example for HAC

- Recalculate distance to update distance matrix

	1,4	2,3	5	6	7
1,4	0.0				
2,3	4.0	0.0			
5	1.4	5.4	0.0		
6	5.8	1.8	7.2	0.0	
7	5.8	1.7	7.3	2.0	0.0

- MIN[dist((1,4),5), (2,3)] = MIN(dist((1,4),(2,3)), (5,(2,3))
  = MIN(4.0, 5.4) = 4.0

- MIN[dist((1,4),5), 6] = MIN(dist((1,4),6), (5,6)) = MIN(5.8, 7.2) = 5.8
- MIN[dist((1,4),5), 7] = MIN(dist((1,4),7), (5,7)) = MIN(5.8, 7.3) = 5.8

Example for HAC

- Updated distance matrix for the cluster ((1,4), 5)

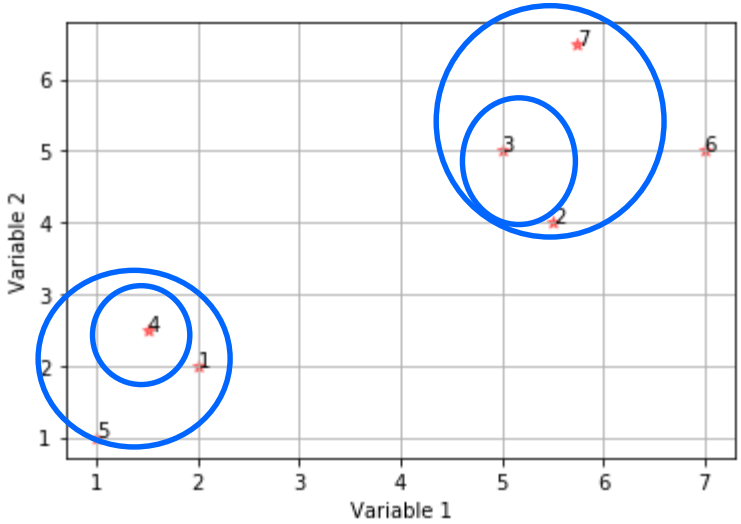
	1,4,5	**2,3**	**6**	**7**
1,4,5	0.0			
2,3	4.0	0.0		
6	5.8	1.8	0.0	
7	5.8	1.7	2.0	0.0

Example for HAC

- Select minimum element to build next cluster formation-

	1,4,5	**2,3**	**6**	**7**
1,4,5	0.0			
2,3	4.0	0.0		
6	5.8	1.8	0.0	
7	5.8	1.7	2.0	0.0

Example for HAC



Example for HAC

- Recalculate distance to update distance matrix

	1,4,5	2,3	6	7
1,4,5	0.0			
2,3	4.0	0.0		
6	5.8	1.8	0.0	
7	5.8	1.7	2.0	0.0

- MIN[dist((2,3),7), (1,4,5)] = MIN(dist((2,3),(1,4,5)), (7,(1,4,5))
= MIN(4.0, 5.8) = 4.0

- MIN[dist((2,3),7), 6] = MIN(dist((2,3),6), (7,6)) = MIN(1.8, 2.0) = 1.8

Example for HAC

- Updated distance matrix for the cluster ((2,3), 7)

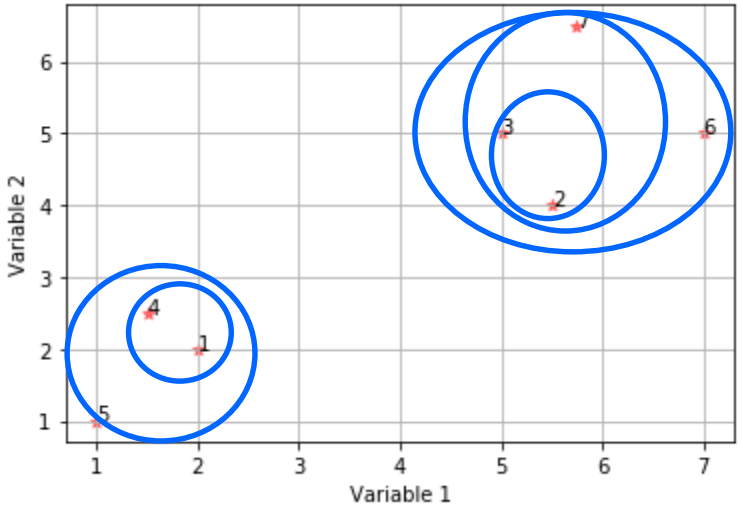
	1,4,5	**2,3,7**	**6**
1,4,5	0.0		
2,3,7	4.0	0.0	
6	5.8	1.8	0.0

Example for HAC

- Select minimum element to build next cluster formation-

	1,4,5	**2,3,7**	**6**
1,4,5	0.0		
2,3,7	4.0	0.0	
6	5.8	1.8	0.0

Example for HAC



Example for HAC

- Recalculate distance to update distance matrix

	1,4,5	2,3,7	6
1,4,5	0.0		
2,3,7	4.0	0.0	
6	5.8	1.8	0.0

- MIN[dist((2,3,7),6), (1,4,5)] = MIN(dist((2,3,7),(1,4,5)), (6,(1,4,5))

= MIN(4.0, 5.8)

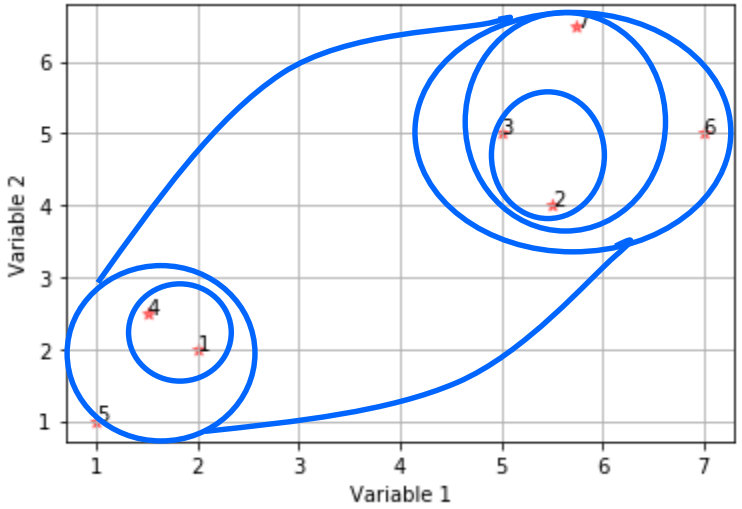
= 4.0

Example for HAC

- Updated distance matrix for the cluster ((2,3,7), 6)

	1,4,5	**2,3,7,6**
1,4,5	0.0	
2,3,7,6	4.0	0.0

Example for HAC



Python demo for HAC

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
from scipy.cluster.hierarchy import fcluster
from scipy.cluster.hierarchy import cophenet
from scipy.spatial.distance import pdist
```

In [2]:
```
data = pd.read_excel("hierarchical_clustering.xlsx")
data
```

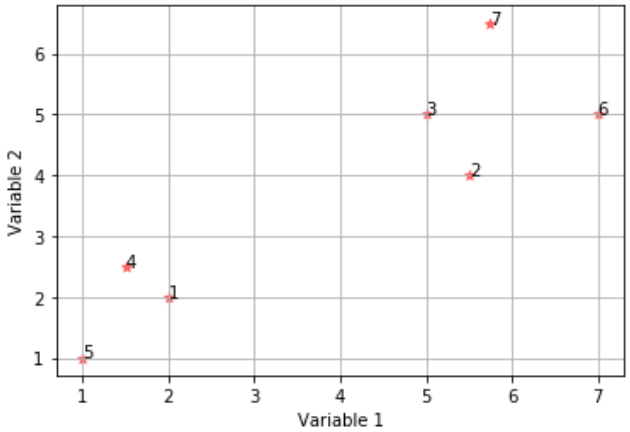
Out[2]:

	Variable 1	Variable 2
0	2.00	2.0
1	5.50	4.0
2	5.00	5.0
3	1.50	2.5
4	1.00	1.0
5	7.00	5.0
6	5.75	6.5

Python demo for HAC

```
In [3]: x = data['Variable 1']
        y = data['Variable 2']
        n = range(1,8)

        fig, ax = plt.subplots()
        ax.scatter(x, y, marker='*', c='red', alpha=0.5)
        plt.grid()
        plt.xlabel("Variable 1")
        plt.ylabel("Variable 2")
        for i, txt in enumerate(n):
            ax.annotate(txt, (x[i], y[i]))
```



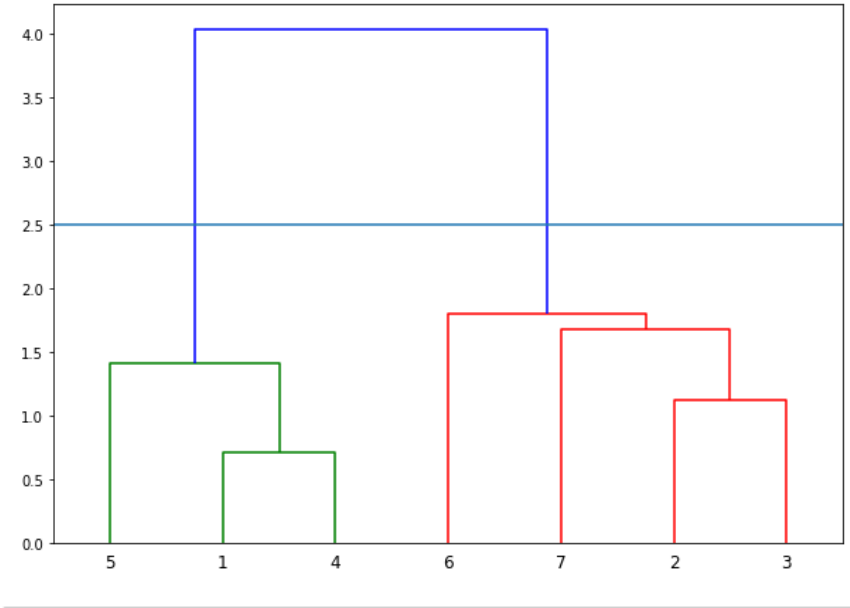
Python demo for HAC

In [4]:
```
from scipy.cluster.hierarchy import dendrogram, linkage

linked = linkage(data, 'single')

labelList = range(1, 8)

plt.figure(figsize=(10, 7))
dendrogram(linked,
           orientation='top',
           labels=labelList,
           distance_sort='descending',
           show_leaf_counts=True)
plt.axhline(y=2.5)
plt.show()
```



Python demo for HAC

In [5]:
```
import sklearn
from sklearn.cluster import AgglomerativeClustering


k=2
Hclustering = AgglomerativeClustering(n_clusters = k, affinity = 'euclidean', linkage = 'single')
Hclustering.fit(data)
```

Out[5]:
```
AgglomerativeClustering(affinity='euclidean', compute_full_tree='auto',
                        connectivity=None, distance_threshold=None,
                        linkage='single', memory=None, n_clusters=2,
                        pooling_func='deprecated')
```

In [6]:
```
Hclustering.fit_predict(data)
```

Out[6]:
```
array([1, 0, 0, 1, 1, 0, 0], dtype=int64)
```

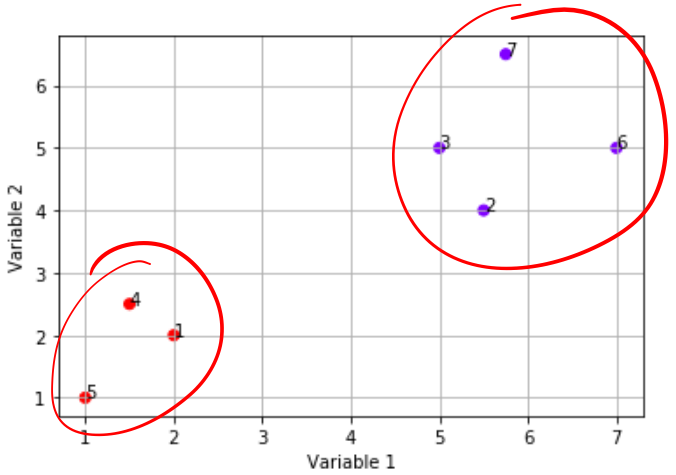
In [7]:
```
print(Hclustering.labels_)
```

```
[1 0 0 1 1 0 0]
```

Python demo for HAC

In [8]:
```
x = data['Variable 1']
y = data['Variable 2']
n = range(1,8)

fig, ax = plt.subplots()
ax.scatter(x, y, c=Hclustering.labels_, cmap='rainbow')
plt.grid()
plt.xlabel("Variable 1")
plt.ylabel("Variable 2")
for i, txt in enumerate(n):
    ax.annotate(txt, (x[i], y[i]))
```



THANK YOU

IIT ROORKEE

NPTEL ONLINE CERTIFICATION COURSE

Classification and Regression Trees (CART - I)

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Introduction to Classification and Regression Trees
- Attribute selection measures – Introduction

Introduction

- Classification is one form of data analysis that can be used to extract models describing important data classes or to predict future data trends
- Classification predicts categorical (discrete, unordered) labels whereas Regression analysis is a statistical methodology that is most often used for numeric (continuous) prediction
- For example, we can build a classification model to categorize bank loan applications as either safe or risky
- Regression model is used to predict expenditures in dollars of potential customers on computer equipment given their income and occupation

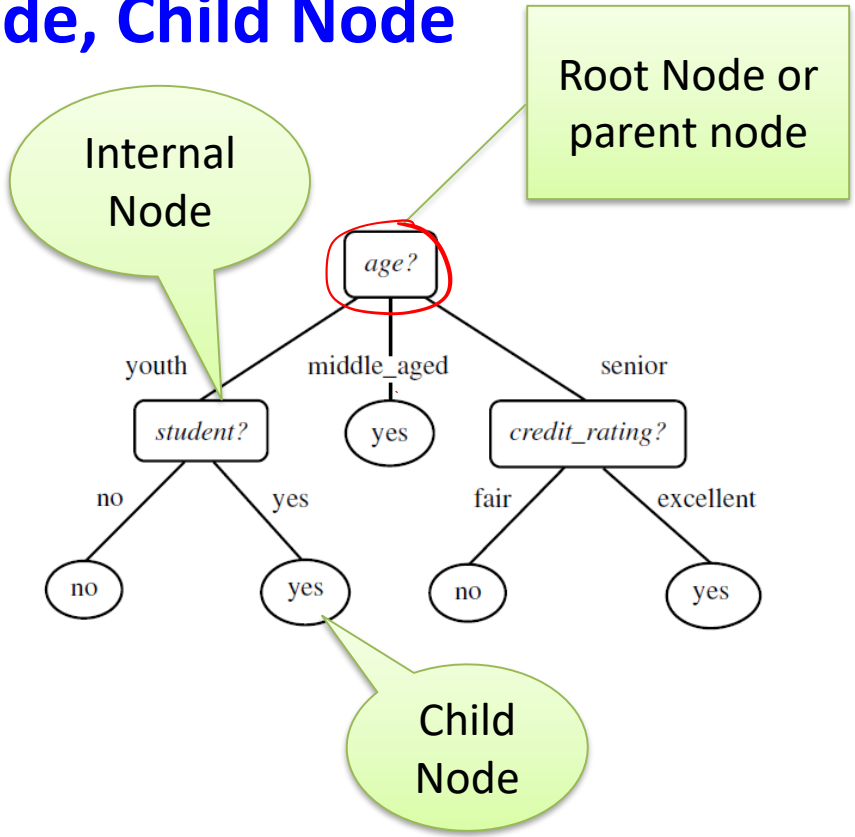
Problem Description for Illustration

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2011. *Data mining: concepts and techniques*. Elsevier.

Root Node, Internal Node, Child Node

- A decision tree uses a tree structure to represent a number of possible *decision paths* and an outcome for each path
- A decision tree consists of root node, internal node and leaf node
- The topmost node in a tree is the **root node** or **parent node**
- It represents entire sample population
- **Internal node** (non-leaf node) denotes a test on an attribute, each branch represents an outcome of the test
- **Leaf node** (or **terminal** node or **child** node) holds a class label
- It can not be further split



Decision Tree Introduction

- A decision tree for the concept *buys_computer*, indicating whether a customer at All Electronics is likely to purchase a computer

- Each internal (non-leaf) node represents a test on an attribute
- Each leaf node represents a class (either *buys_computer = yes* or *buys computer = no*).

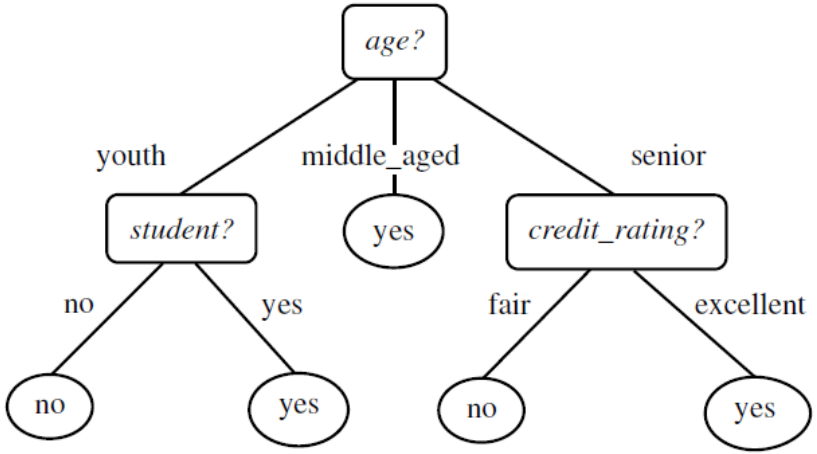


Figure 1.1 : Decision Tree

CART Introduction

- CART comes under supervised learning technique
- CART adopt a greedy(i.e., non backtracking) approach in which decision trees are constructed in a top-down recursive divide-and-conquer manner
- It is very interpretable model

Decision Tree Algorithm

Input:

- Data partition, D, which is a set of training tuples and their associated class labels;
- Attribute list, the set of candidate attributes;
- Attribute selection method, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a splitting attribute and, possibly, either a split point or splitting subset.

Output: **A decision tree**

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Decision Tree Algorithm

- The algorithm is called with three parameters: D, attribute list, and Attribute selection method
- D is defined as a data partition. Initially, it is the complete set of training tuples and their associated class labels
- The parameter attribute list is a list of attributes or independent variables which are describing the tuples
- Attribute selection method specifies a heuristic procedure for selecting the attribute that “best” discriminates the given tuples according to class

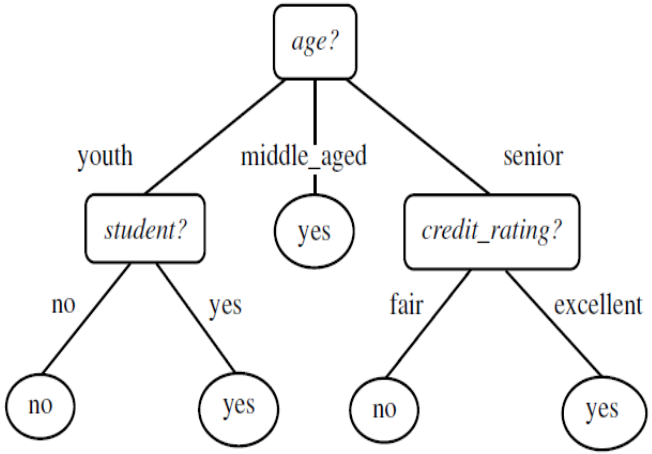
Decision Tree Algorithm

- This procedure employs an attribute selection measure, such as information gain, gain ratio or the Gini index.
- Whether the tree is strictly binary is generally driven by the attribute selection measure
- Some attribute selection measures, such as the Gini index, enforce the resulting tree to be binary. Others, like information gain, do not, therein allowing multiway splits (i.e., two or more branches to be grown from a node).

Decision Tree Method

Method:

(1) create a node N;
(2) **if** tuples in D are all of the same class, C **then**
(3) return N as a leaf node labeled with the class C;
(4) **if** *attribute_list* is empty **then**
(5) return N as a leaf node labeled with the majority class in D; // majority voting
(6) apply **Attribute_selection_method**(D, *attribute_list*) to **find** the "best" *splitting_criterion*;
(7) label node N with *splitting_criterion*;
(8) **if** *splitting_attribute* is discrete-valued **and**
multiway splits allowed **then** // not restricted to binary trees
(9) *attribute_list* ← *attribute_list* − *splitting_attribute*; // remove *splitting_attribute*
(10) **for each** outcome j of *splitting_criterion*
// partition the tuples and grow subtrees for each partition
(11) let D_j be the set of data tuples in D satisfying outcome j; // a partition
(12) **if** D_j is empty **then**
(13) attach a leaf labeled with the majority class in D to node N;
(14) **else** attach the node returned by **Generate_decision_tree**(D_j, *attribute_list*) to node N;
endfor
(15) return N;



N-Node
C- Class
D- tuples in training data set

Decision Tree Method step 1 to 6

- The tree starts as a single node, N, representing the training tuples in D (step 1).
- If the tuples in D are all of the same class, then node N becomes a leaf and is labelled with that class (steps 2 and 3)
- Steps 4 and 5 are terminating conditions
- Otherwise, the algorithm calls Attribute selection method to determine the splitting criterion
- The splitting criterion (like Gini) tells us which attribute to test at node N by determining the "best" way to separate or partition the tuples in D into individual classes (step 6)

Method:

(1) create a node N;
(2) **if** tuples in D are all of the same class, C **then**
(3) return N as a leaf node labeled with the class C;
(4) **if** *attribute_list* is empty **then**
(5) return N as a leaf node labeled with the majority class in D; // majority voting
(6) apply **Attribute_selection_method**(D, *attribute_list*) to **find** the "best" *splitting_criterion*;
(7) label node N with *splitting_criterion*;
(8) **if** *splitting_attribute* is discrete-valued **and**
multiway splits allowed **then** // not restricted to binary trees
(9) *attribute_list* ← *attribute_list* − *splitting_attribute*; // remove *splitting_attribute*
(10) **for each** outcome j of *splitting_criterion*
// partition the tuples and grow subtrees for each partition
(11) let D_j be the set of data tuples in D satisfying outcome j; // a partition
(12) **if** D_j is empty **then**
(13) attach a leaf labeled with the majority class in D to node N;
(14) **else** attach the node returned by **Generate_decision_tree**(D_j, *attribute_list*) to node N;
endfor
(15) return N;

Decision Tree Method - Step 7 - 11

- The splitting criterion indicates the splitting attribute and may also indicate either a split-point or a splitting subset
- The splitting criterion is determined so that, ideally, the resulting partitions at each branch are as "pure" as possible. A partition is pure if all of the tuples in it belong to the same class.
- The node N is labelled with the splitting criterion, which serves as a test at the node (step 7).
- A branch is grown from node N for each of the outcomes of the splitting criterion.
- The tuples in D are partitioned accordingly (steps 10 to 11)

Method:

(1) create a node N;
(2) **if** tuples in D are all of the same class, C **then**
(3) return N as a leaf node labeled with the class C;
(4) **if** *attribute_list* is empty **then**
(5) return N as a leaf node labeled with the majority class in D; // majority voting
(6) apply **Attribute_selection_method**(D, *attribute_list*) to **find** the "best" *splitting_criterion*;
(7) label node N with *splitting_criterion*;
(8) **if** *splitting_attribute* is discrete-valued **and**
multiway splits allowed **then** // not restricted to binary trees
(9) *attribute_list* ← *attribute_list* − *splitting_attribute*; // remove *splitting_attribute*
(10) **for each** outcome j of *splitting_criterion*
// partition the tuples and grow subtrees for each partition
(11) let D_j be the set of data tuples in D satisfying outcome j; // a partition
(12) **if** D_j is empty **then**
(13) attach a leaf labeled with the majority class in D to node N;
(14) **else** attach the node returned by **Generate_decision_tree**(D_j, *attribute_list*) to node N;
endfor
(15) return N;

Three possibilities for partitioning tuples based on the splitting criterion

- There are three possible scenarios, as illustrated in Figure (a), (b) and (c).
- Let A be the splitting attribute. A has ‘v’ distinct values,{a1,a2,...,av}, based on the training data
- If A is discrete-valued in figure (a), then one branch is grown for each known value of A.

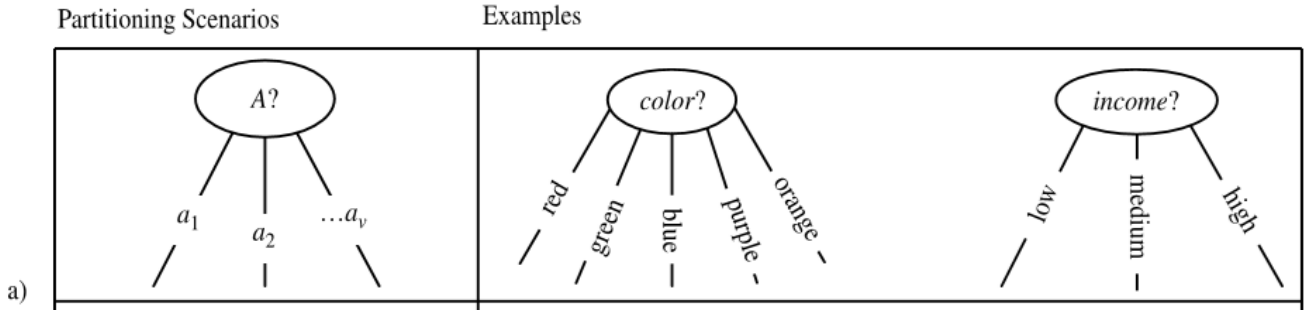


Figure (a)

Three possibilities for partitioning tuples based on the splitting criterion

- If A is continuous-valued in figure (b), then two branches are grown, corresponding to A ≤ split point and A > split point.
- Where split point is the split-point returned by Attribute selection method as part of the splitting criterion.

Figure (b)

Three possibilities for partitioning tuples based on the splitting criterion

- If A is discrete-valued and a binary tree must be produced, then the test is of the form A ∈ S_A, where S_A is the splitting subset for A.

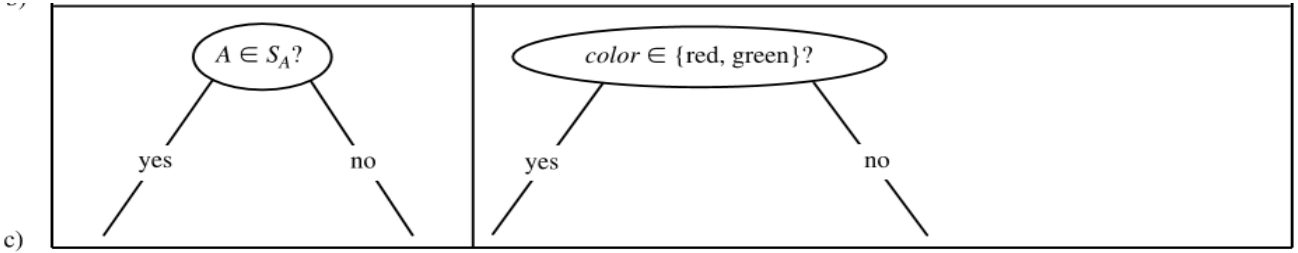


Figure (c)

Decision Tree Method – termination condition

- The algorithm uses the same process recursively to form a decision tree for the tuples at each resulting partition, D_j, of D (step 14).

- The recursive partitioning stops only when anyone of the following terminating conditions is true:

1. All of the tuples in partition D (represented at node N) belong to the same class (steps 2 and 3), or

Decision Tree Method – termination condition

2. There are no remaining attributes on which the tuples may be further partitioned (step4).
   - In this case, majority voting is employed(step 5).
   - This involves converting node N into a leaf and labelling it with the most common class in D.
   - Alternatively, the class distribution of the node tuples may be stored.
3. There are no tuples for a given branch, that is, a partition Dj is empty (step 12).
   - In this case, a leaf is created with the majority class in D (step 13).
   - The resulting decision tree is returned (step 15).

Attribute Selection Measures

- Attribute selection measures are also known as splitting rules because they determine how the tuples at a given node are to be split
- It is a heuristic approach for selecting the splitting criterion that “best” separates a given data partition, D, of class-labeled training tuples into individual classes
- The attribute selection measure provides a ranking for each attribute describing the given training tuples
- The attribute having the best score for the measure is chosen as the splitting attribute for the given tuples

Attribute Selection Measures

- If the splitting attribute is continuous-valued or if we are restricted to binary trees then, respectively, either a ‘split point’ or a ‘splitting subset’ must also be determined as part of the splitting criterion

- There are three popular attribute selection measures
  - information gain,
  - gain ratio, and
  - Gini index

- CART algorithm uses information gain and Gini index measure for attribute selection

Attribute Selection Measures

The notation used herein is as follows. Let D, the data partition, be a training set of class-labeled tuples. Suppose the class label attribute has m distinct values defining m distinct classes, C_i (for $i = 1, \ldots, m$). Let $C_{i,D}$ be the set of tuples of class C_i in D. Let $|D|$ and $|C_{i,D}|$ denote the number of tuples in D and $C_{i,D}$, respectively.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

$m = 2$

Information Gain

- This measure studied the value or “information content” of messages
- The attribute with the highest information gain is chosen as the splitting attribute for node
- This attribute minimizes the information needed to classify the tuples in the resulting partitions and reflects the least randomness or “impurity” in these partitions
- This approach minimizes the expected number of tests needed to classify a given tuple

Information Gain-Entropy Measure

- The expected information needed to classify a tuple in *D* is given by

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

- Where p_i is the probability that an arbitrary tuple in *D* belongs to class C_i and is estimated by $|C_{i,D}|/|D|$.
- A log function to the base 2 is used, because the information is encoded in bits
- Info(D) (or **Entropy** of D)is just the average amount of information needed to identify the class label of a tuple in *D*

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

Attribute Selection Measures

- It is quite likely that the partitions will be impure (e.g., where a partition may contain a collection of tuples from different classes rather than from a single class).
- How much more information would we still need (after the partitioning) in order to arrive at an exact classification?
- This amount is measured by

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

- The term $|D_j|$ / $|D|$ acts as the weight of the j_{th} partition. $Info_A(D)$ is the expected information required to classify a tuple from *D* based on the partitioning by *A*.

Information Gain

- The smaller the expected information (still) required, the greater the purity of the partitions
- Information gain is defined as the difference between the original information requirement (i.e., based on just the proportion of classes) and the new requirement (i.e., obtained after partitioning on *A*). That is,

$$Gain(A) = Info(D) - Info_A(D)$$

- The attribute A with the highest information gain, (*Gain*(*A*)), is chosen as the splitting attribute at node N.

Gini Index

- Gini index is used to measures the impurity of D, a data partition or set of training tuples, as

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

- Where p_i is the probability that a tuple in Dbelongs to class C_i and is estimated by $|C_{i,D}|/|D|$.
- The sum is computed over 'm'classes.
- The Gini index considers a binary split for each attribute

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

Gini Index

- When considering a binary split, we compute a weighted sum of the impurity of each resulting partition
- For example, if a binary split on Apartitions Dinto D_1 and D_2 , the Gini index of Dgiven that partitioning is-

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

- For each attribute, each of the possible binary splits is considered
- For a discrete-valued attribute, the subset that gives the minimum Gini index for that attribute is selected as its splitting subset

Gini Index

- For continuous-valued attributes, each possible split-point must be considered
- The strategy is similar where the midpoint between each pair of (sorted) adjacent values is taken as a possible split-point.
- For a possible split-point of A, D_1 is the set of tuples in D satisfying A ≤ split point, and D_2 is the set of tuples in D satisfying A > split point.
- The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- The attribute that maximizes the reduction in impurity (or, equivalently, has the minimum Gini index) is selected as the splitting attribute

Which attribute selection measure is the best?

- All measures have some bias.
- The time complexity of decision tree generally increases exponentially with tree height
- Hence, measures that tend to produce shallower trees (e.g., with multiway rather than binary splits, and that favour more balanced splits) may be preferred.
- However, some studies have found that shallow trees tend to have a large number of leaves and higher error rates
- Several comparative studies suggests no one attribute selection measure has been found to be significantly superior to others.

Tree Pruning

- When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers
- Tree pruning use statistical measures to remove the least reliable branches
- Pruned trees tend to be smaller and less complex and, thus, easier to comprehend
- They are usually faster and better at correctly classifying independent test data than unpruned trees

How does Tree Pruning Work?

- There are two common approaches to tree pruning: **pre-pruning** and **post-pruning**.
- In the **pre-pruning** approach, a tree is "pruned" by halting its construction early (e.g., by deciding not to further split or partition the subset of training tuples at a given node).
- When constructing a tree, measures such as statistical significance, information gain, Gini index can be used to assess the goodness of a split.

How does Tree Pruning Work?

- The **post-pruning** approach removes sub_trees from a “fully grown” tree
- A subtree at a given node is pruned by removing its branches and replacing it with a leaf
- The leaf is labelled with the most frequent class among the subtree being replaced
- For example, the subtree at node “A3?” in the unpruned tree of Figure 1.2
- The most common class within this subtree is “class B”
- In the pruned version of the tree, the subtree in question is pruned by replacing it with the leaf “class B

How does Tree Pruning Work?

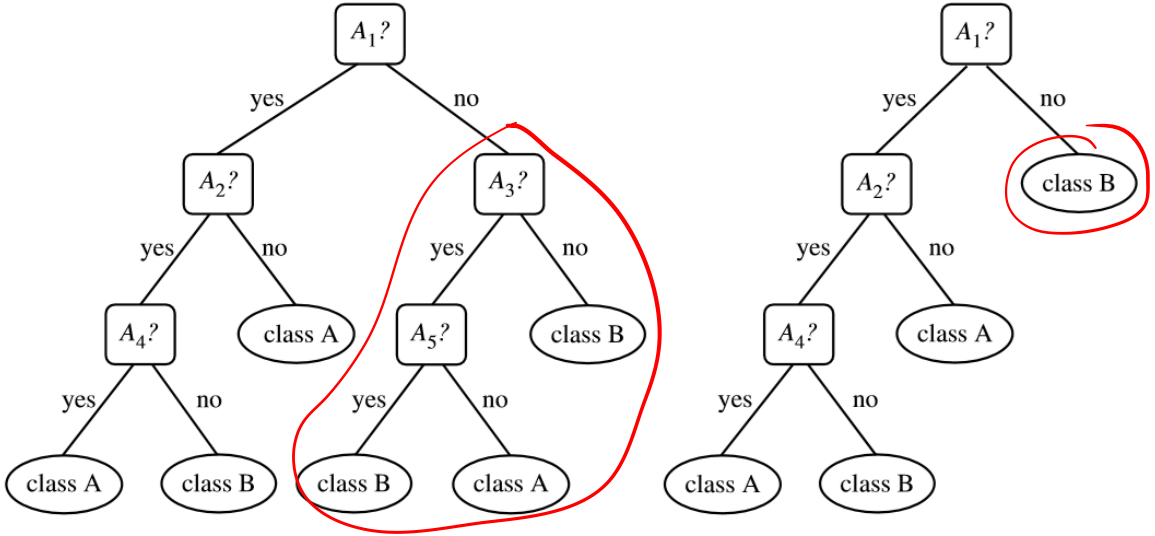


Figure: 1.2 An unpruned decision tree and a post-pruned decision tree

THANK YOU

IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Measures of Attribute Selection

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Measures of attribute selection using
  - Information Gain
  - Gain ratio
  - Gini Index

Example

- The following Table presents a training set, D, of class-labeled tuples randomly selected from the AllElectronics customer database

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2011. *Data mining: concepts and techniques*. Elsevier.

Example

- In this example, each attribute is discrete-valued
- Continuous-valued attributes have been generalized
- The class label attribute, buys computer, has two distinct values (namely, {yes, no}); therefore, there are two distinct classes (that is, m = 2)
- Let class C_1 correspond to 'yes' and class C_2 correspond to 'no'.
- There are nine tuples of class 'yes' and five tuples of class 'no'.
- A (root) node N is created for the tuples in D

Expected information needed to classify a tuple in *D*

- To find the splitting criterion for these tuples, we must compute the information gain of each attribute
- Let us consider Class: buys computer as decision criteria D
- Calculate information:
- = $-p_y \log_2 (p_y) - p_n \log_2 (p_n)$
- Where p_y is probability of 'yes' and p_n is probability of 'no'

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

Calculation of entropy for ‘ Youth’

- Age can be:
  - youth
  - Middle_aged
  - Senior
- Youth

Youth	Class: buys computer
Yes	2
No	3

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculation of entropy for ‘ Youth’

- Calculate Entropy for youth:
- Entropy for youth = $-\frac{2}{5}\log_2\frac{2}{5}-\frac{3}{5}\log_2\frac{3}{5}$

- Middle_aged

middle	Class: buys computer
Yes	4
No	0

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculation of entropy for ‘ Middle Age’

- Calculate Entropy for middle_aged
- $= -\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4}$
- $= 0$

- For Senior

Senior	Class: buys computer
Yes	3
No	2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for senior

Calculate Entropy for senior

= $-\frac{3}{5}\log_2\frac{3}{5}-\frac{2}{5}\log_2\frac{2}{5}$

The expected information needed to classify a tuple in D according to age

The expected information needed to classify a tuple in D if the tuples are partitioned according to age is

$$
\begin{aligned}
Info_{age}(D) = & \frac{5}{14} \times \left(-\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}\right) \\
& + \frac{4}{14} \times \left(-\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4}\right) \\
& + \frac{5}{14} \times \left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}\right) \\
= & \ 0.694 \text{ bits.}
\end{aligned}
$$

Calculation information Gain of Age

- Gain of Age:

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

Calculation information Gain of Income

- Calculation of gain for income:
- Income cane be:
  - High
  - Medium
  - Low

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

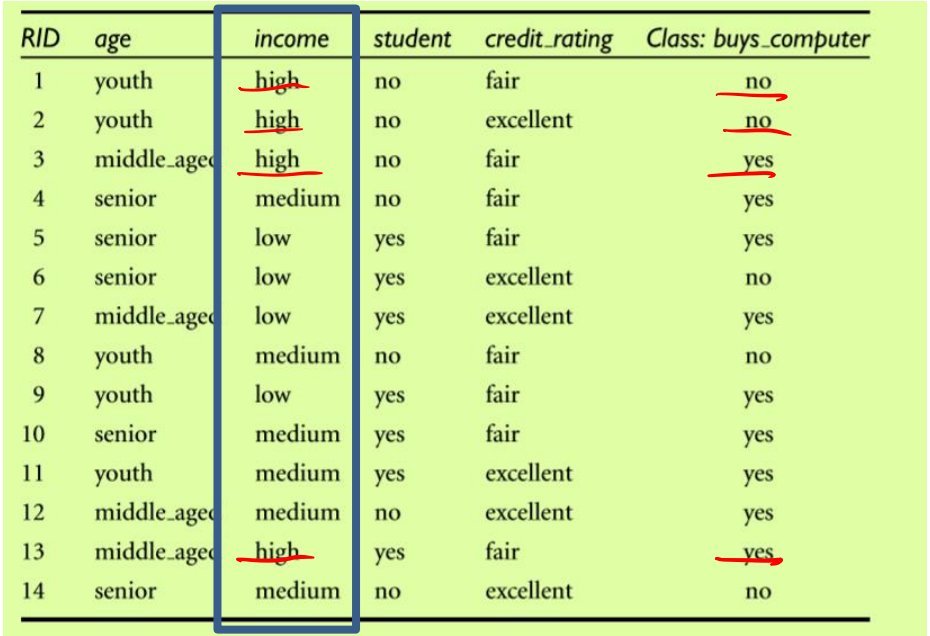
Calculate Entropy for high

- High :

High	Class: buys computer
Yes	2
No	2

- Calculate Entropy for high:

  $= -(2/4)\log_2(2/4) - (2/4)\log_2(2/4)$



RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for ‘medium’

- Medium:

Medium	Class: buys computer
Yes	4
No	2

- Calculate Entropy for Medium:

  $= -(4/6)\log_2(4/6) - (2/6)\log_2(2/6)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for ‘low’

- Low :

Low	Class: buys computer
No	1
Yes	3

- Calculate Entropy for Low:

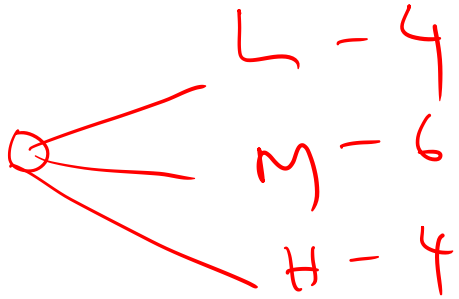
  $= -(1/4)\log_2(1/4) - (3/4)\log_2(3/4)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gain of income

- The expected information needed to classify a tuple in D if the tuples are partitioned according to income is:
- $Info_{income}(D) = (4/14)(-(2/4)\log_2(2/4) - (2/4)\log_2(2/4)) +$
  $(6/14)(-(4/6)\log_2(4/6) - (2/6)\log_2(2/6)) +$
  $(4/14)(-(1/4)\log_2(1/4) - (3/4)\log_2(3/4))$
  $= 0.911$

Gain of income : $Info(D) - Info_{income}(D)$
$= 0.94 - 0.911 = 0.029$



Calculation of gain for student

- Calculation of gain for student
- Student can be:
  - Yes (7)
  - No (7)

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for No

- No :

No	Class: buys computer
Yes	3
No	4

- Calculate Entropy for No:

  $= -(3/7)\log_2(3/7) - (4/7)\log_2(4/7)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for ‘Yes’

- Yes :

Yes	Class: buys computer
Yes	6
No	1

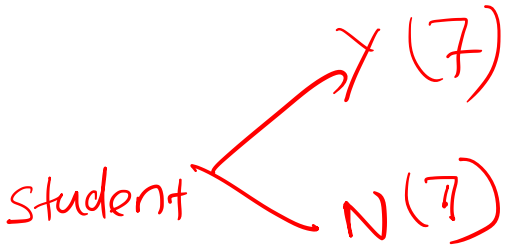
- Calculate Entropy for Yes:

  $= -(6/7)\log_2(6/7) - (1/7)\log_2(1/7)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gain of student

- The expected information needed to classify a tuple in D if the tuples are partitioned according to student is:
- $\text{Info}_{\text{Student}}(D) = (7/14)(-(3/7)\log_2(3/7) - (4/7)\log_2(4/7)) +$
  $(7/14)(-(6/7)\log_2(6/7) - (1/7)\log_2(1/7))$
  $=0.789$
  - Gain(student) :
    $\text{Info}(D) - \text{Info}_{\text{student}}(D)$
    $= 0.94 - 0.789 = 0.151$



Calculation of gain for credit rating

- Calculation of gain for credit rating
- Credit rating can be:
  - Fair – 8
  - Excellent - 6

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for Fair

- Fair :

Fair	Class: buys computer
Yes	6
No	2

- Calculate Entropy for Fair:

  $= -(6/8)\log_2(6/8) - (2/8)\log_2(2/8)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for Excellent

- Excellent :

Yes	Class: buys computer
Yes	3
No	3

- Calculate Entropy for Excellent:

  $= -(3/6)\log_2(3/6) - (3/6)\log_2(3/6)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gain for credit rating

- The expected information needed to classify a tuple in D if the tuples are partitioned according to Credit rating is:
- $\text{Info}_{\text{Credit rating}}(D) = (8/14)\,(-(6/8)\log_2(6/8) - (2/8)\log_2(2/8)) +$
  $(6/14)\,(-(3/6)\log_2(3/6) - (3/6)\log_2(3/6))$
  $=0.892$
- Gain for credit rating :

  $\text{Info}(D) - \text{Info}_{\text{Credit rating}}(D)$

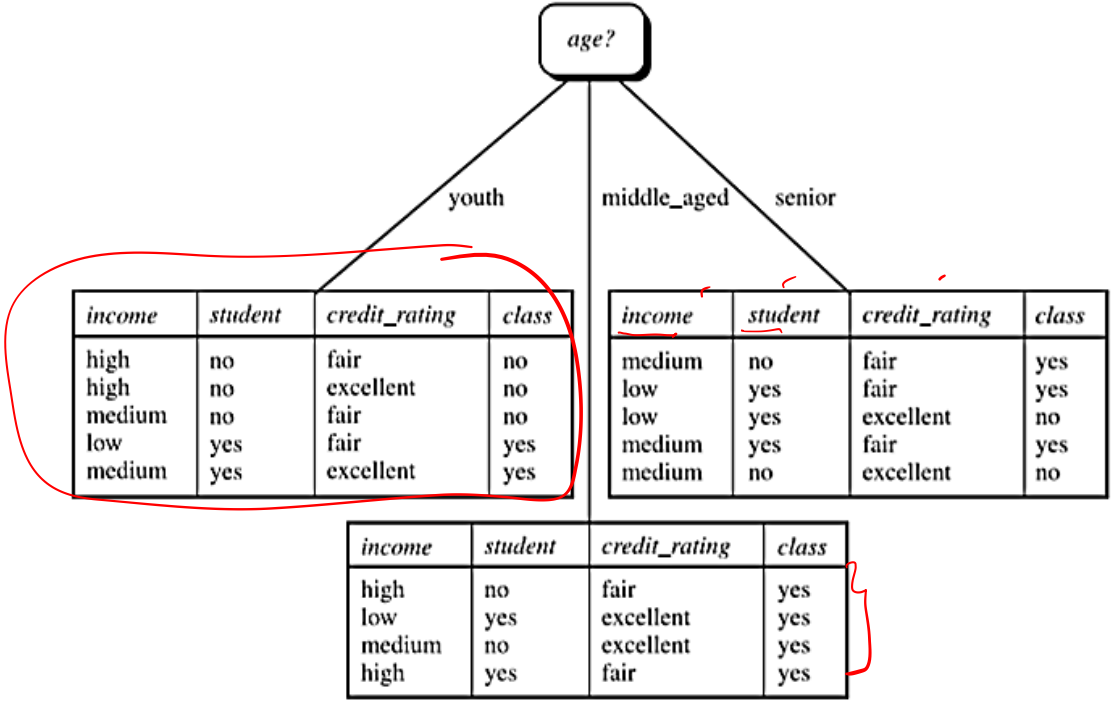
  $= 0.94 - 0.892 = 0.048$

Independent variable	Information gain
Age	0.246
Income	0.029
Student	0.151
Credit_rating	0.048

Selection of root classifier

- Because age has the highest information gain among the attributes, it is selected as the splitting attribute
- Node N is labelled with age, and branches are grown for each of the attribute's values
- The tuples are then partitioned accordingly
- Notice that the tuples falling into the partition for age = middle aged all belong to the same class
- Because they all belong to class "yes," a leaf should therefore be created at the end of this branch and labelled with "yes."

Decision tree



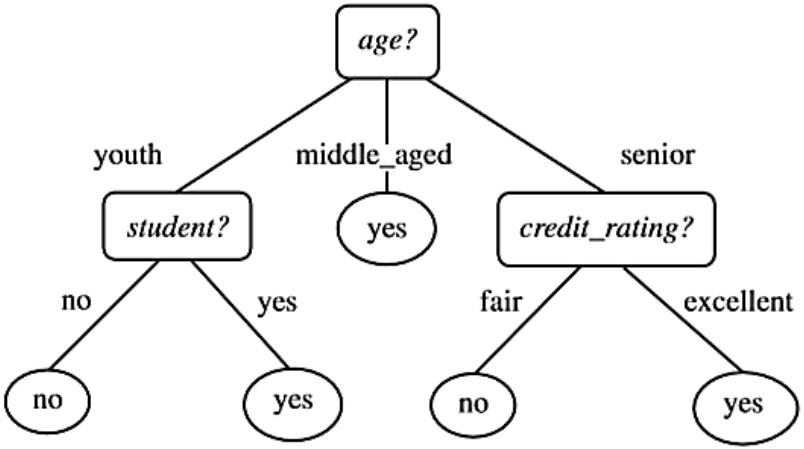
income	*student*	*credit_rating*	*class*
high	no	fair	no
high	no	excellent	no
medium	no	fair	no
low	yes	fair	yes
medium	yes	excellent	yes

income	*student*	*credit_rating*	*class*
medium	no	fair	yes
low	yes	fair	yes
low	yes	excellent	no
medium	yes	fair	yes
medium	no	excellent	no

income	*student*	*credit_rating*	*class*
high	no	fair	yes
low	yes	excellent	yes
medium	no	excellent	yes
high	yes	fair	yes

Decision tree

- The final decision tree returned by the algorithm is shown in Figure



Thank You

IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Attribute selection Measures in CART : II

Dr. A. Ramesh

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

- Attribute selection measures:
  - Gain Value
  - Gain ratio
  - Gini Index

Gain Ratio

- The information gain measure is biased toward tests with many outcomes
- That is, it prefers to select attributes having a large number of values
- For example, consider an attribute that acts as a unique identifier, such as product ID.
- A split on product ID would result in a large number of partitions (as many as there are values), each one containing just one tuple

Gain Ratio

- Because each partition is pure, the information required to classify dataset D based on this partitioning would be $\text{Info}_{\text{product ID}}(D) = 0$
- Information Gain = Info D-$\text{Info}_{\text{product ID}}(D)$ = maximum
- Therefore, the information gained by partitioning on this attribute is maximal
- Clearly, such a partitioning is useless for classification
- Gain ratio is an extension to information gain which attempts to overcome this bias

Split information

- It applies a kind of normalization to information gain using a “split information” value defined analogously with Info(D) as:

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times \log_2\left(\frac{|D_j|}{|D|}\right)$$

- D_j= single partion
- D = Data set
- This value represents the potential information generated by splitting the training data set, D, into v partitions, corresponding to the v outcomes of a test on attribute A

Gain ratio

- Gain ratio differs from information gain, which measures the information with respect to classification that is acquired based on the same partitioning
- The gain ratio is defined as

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

- The attribute with the maximum gain ratio is selected as the splitting attribute

Gain Ratio example

- Consider the previous example for computation of gain ratio for the attribute income
- A test on income splits the data of the following Table into three partitions, namely low, medium, and high, containing four, six, and four tuples,respectively

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2011. *Data mining: concepts and techniques*. Elsevier.

Calculate Entropy for high

- High :

High	Class: buys computer
Yes	2
No	2

- Calculate Entropy for high:

  $= -(2/4)\log_2(2/4) - (2/4)\log_2(2/4)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for ‘medium’

- Medium:

Medium	Class: buys computer
Yes	4
No	2

- Calculate Entropy for Medium:

  $= -(4/6)\log_2(4/6) - (2/6)\log_2(2/6)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for ‘low’

- Low :

Low	Class: buys computer
Yes	3
No	1

- Calculate Entropy for Low:

  $= - (3/4)\log_2(3/4) - (1/4)\log_2(1/4)$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Calculate Entropy for buying class D

- Calculate information:
- = $-p_y \log_2 (p_y) - p_n \log_2 (p_n)$
- Where p_y is probability of yes and p_n is probability of no

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

Gain of income

- The expected information needed to classify a tuple in D if the tuples are partitioned according to income is:
- $\text{Info}_{\text{income}}(D) = (4/14)\,(-(2/4)\log_2(2/4) - (2/4)\log_2(2/4)) +$
  $(6/14)\,(-(4/6)\log_2(4/6) - (2/6)\log_2(2/6)) +$
  $(4/14)\,(-(1/4)\log_2(1/4) - (3/4)\log_2(3/4))$
  $= 0.911$ bits

Gain of income : $\text{Info}(D) - \text{Info}_{\text{income}}(D)$
$= 0.94 - 0.911 = \boxed{0.029}$

Gain-Ratio(income)

- Calculation of split ratio:

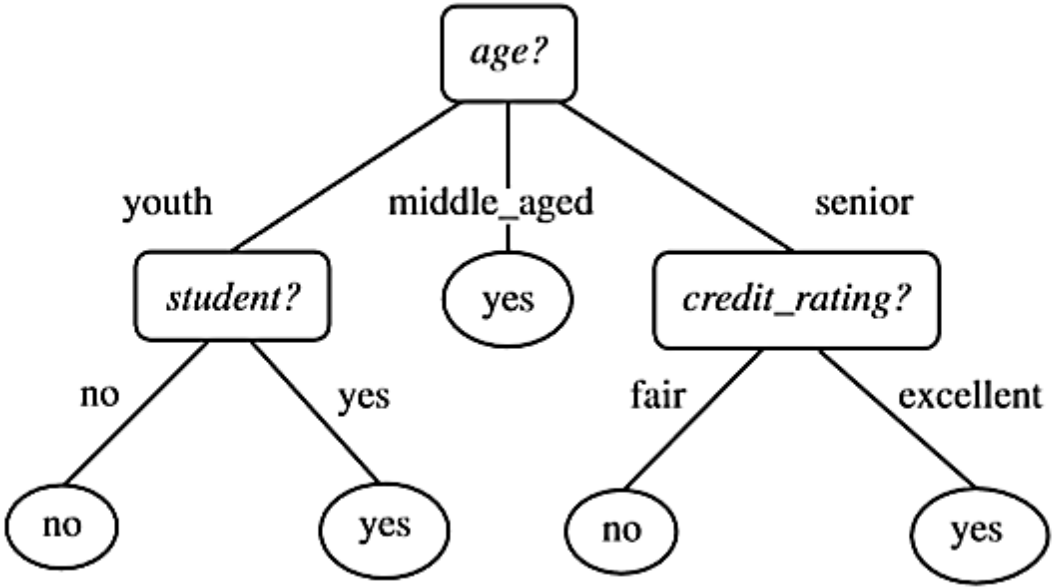
$$SplitInfo_A(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right)$$

$$= 0.926.$$

- Therefore, Gain-Ratio(income) = 0.029/0.926 = 0.031

Interpretation

- Further we calculate the same for the rest 3 criteria (age, student, credit rating)
- The one with maximum Gain ratio value will results in the maximum reduction in impurity of the tuples in D and is returned as the splitting criterion


age?
youth
middle_aged
senior
student?
yes
credit_rating?
no
yes
fair
excellent
no
yes
no
yes

Decision tree using Gini index

- Let's take the Introduction of a decision tree using Gini index
- Let D be the training data of the following table

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2011. *Data mining: concepts and techniques*. Elsevier.

Example

- In this example, each attribute is discrete-valued
- Continuous-valued attributes have been generalized
- The class label attribute, buys computer, has two distinct values (namely, {yes, no}); therefore, there are two distinct classes (that is, m = 2)
- Let class C_1 correspond to 'yes' and class C_2 correspond to 'no'.
- There are nine tuples of class 'yes' and five tuples of class 'no'.
- A (root) node N is created for the tuples in D

Calculation of Gini(D)

- We first use the following Equation for Gini index to compute the impurity of D:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2,$$

$$= \quad Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- Lets calculate Gini index for income attribute
- To find the splitting criterion for the tuples in D, we need to compute the Gini index for each attribute
- Let's start with the attribute income and consider each of the possible splitting subsets
- Income has three possible values, namely {low, medium, high}, then the possible subsets are {low, medium, high}, {low, medium}, {low, high}, {medium, high}, {low}, {medium}, {high}, and {}
- Power set and empty set will not be used for splitting

Gini index for income attribute

- Consider the subset{low, medium}
- This would result in 10 tuples in partition D1 satisfying the condition "income ∈{low, medium}"
- The remaining four tuples of D (high) would be assigned to partition D2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D1

- Low + Medium:

Medium + Low	Class: buys computer
Yes	3+4 =7
No	1+ 2 = 3

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D2

- High : D_y

High	Class: buys computer
Yes	2
No	2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

$$Gini_{income \in \{low,medium\}}(D)$$

$$= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2)$$

$$= (10/14)\,(1 - (7/10)^2 - (3/10)^2) +$$

$$(4/14)\,(1 - (2/4)^2 - (2/4)^2)$$

$$= 0.443 = Gini_{income \in \{high\}}$$

Gini index for income attribute

- Consider the subset{high, medium}
- This would result in 10 tuples in partition D1 satisfying the condition “income ∈{high, medium}”
- The remaining four tuples of D (low) would be assigned to partition D_2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D1

- High + Medium:

Medium + high	Class: buys computer
Yes	2+4
No	2 + 2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D2

- Low :

Low	Class: buys computer
No	1
Yes	3

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

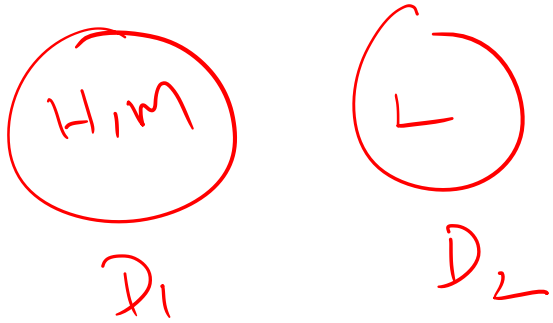
Gini $_{\text{income} \in \{\text{high, medium}\}}$

$$= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2)$$

$$= (10/14)\,(1 - (6/10)^2 - (4/10)^2) +$$

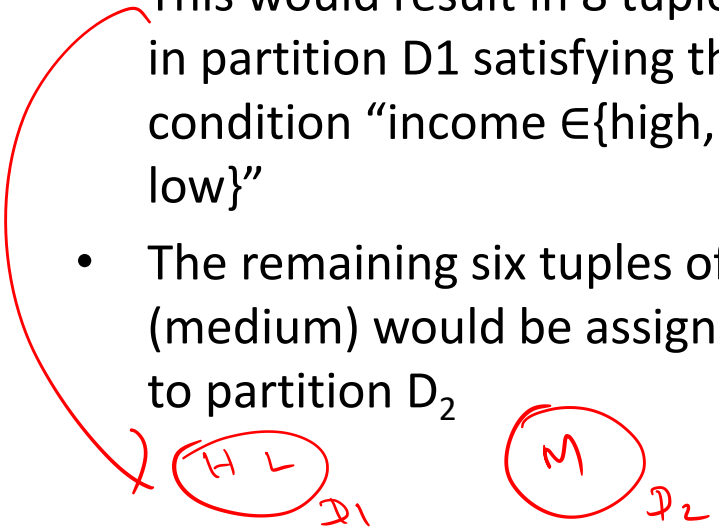
$$(4/14)\,(1 - (1/4)^2 - (3/4)^2)$$

$$= 0.45 = \text{Gini}_{\text{income} \in \{\text{low}\}}$$



Gini index for income attribute

- Consider the subset{high, low}
- This would result in 8 tuples in partition D1 satisfying the condition “income ∈{high, low}”
- The remaining six tuples of D (medium) would be assigned to partition D_2



RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D1

- High + low:

high + low	Class: buys computer
Yes	2+3
No	2 + 1

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D2

- Medium:

Low	Class: buys computer
No	2
Yes	4

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

  $Gini_{income \in \{high, low\}}$

$$= (8/14)\,(1- (5/8)^2 - (3/8)^2) + (6/14)\,(1- (2/6)^2 - (4/6)^2)$$

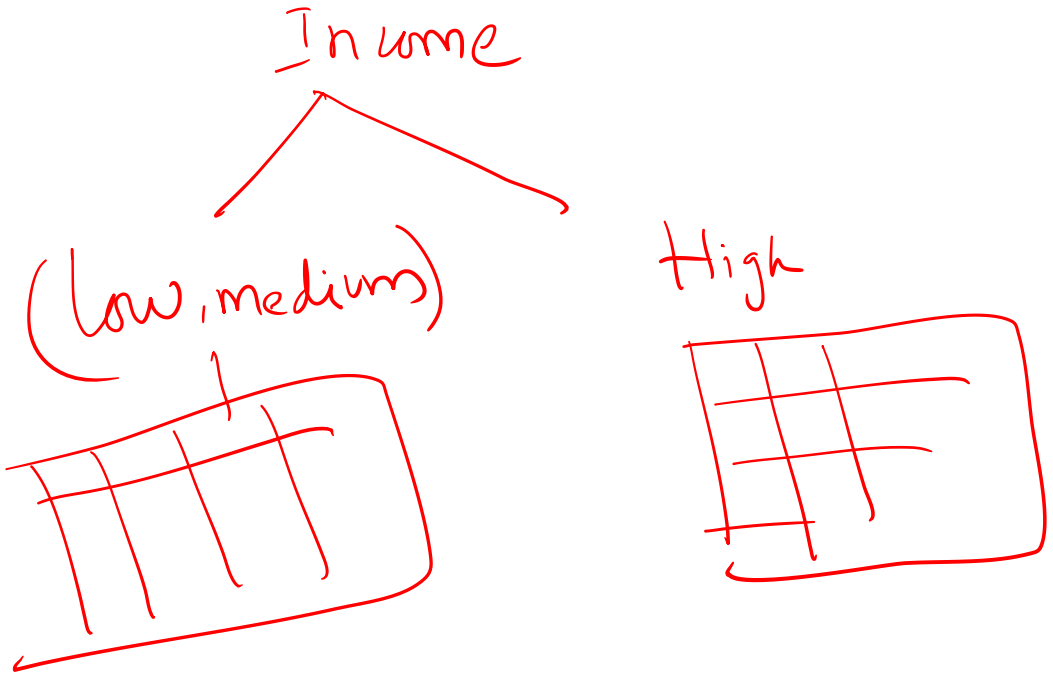
$$= 0.458 = Gini_{income \in \{medium\}}$$

Gini Index values

	Gini Index values
Gini $_{\text{income} \in \{\text{high, low}\}}$	0.458
Gini $_{\text{income} \in \{\text{high, medium}\}}$	0.45
Gini $_{\text{income} \in \{\text{medium, low}\}}$	0.443

Interpretation

- The best binary split for attribute income is on {medium, low} (or {high}) because it minimizes the Gini index
- The splitting subset {medium,low} therefore give the minimum Gini index for attribute income
- Reduction in impurity = 0.459 – 0.443 = 0.016
- Further we calculate the same for the rest 3 criteria (age, student, credit rating)
- The one with minimum Gini index value will results in the maximum reduction in impurity of the tuples in D and is returned as the splitting criterion


Income
(low, medium)
High

Thank You

IIT ROORKEE

FREE ONLINE EDUCATION swayam शिक्षित भारत, उन्नत भारत

NPTEL ONLINE CERTIFICATION COURSE

Classification and Regression Trees (CART – III)

Dr A. RAMESH

DEPARTMENT OF MANAGEMENT STUDIES

Agenda

Python demo for CART model -

- Visualizing Decision Tree
- Interpretation of CART model

Example

Problem Description-

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2011. *Data mining: concepts and techniques*. Elsevier.

Import Relevant Libraries and Loading Data File

In [1]:
```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
```

In [2]:
```
1 data = pd.read_excel('CART.xlsx')
```

In [3]:
```
1 data
```

Out[3]:

	RID	age	income	student	credit_rating	buys_computer
0	1	youth	high	no	fair	no
1	2	youth	high	no	excellent	no
2	3	middle_aged	high	no	fair	yes
3	4	senior	medium	no	fair	yes
4	5	senior	low	yes	fair	yes
5	6	senior	low	yes	excellent	no
6	7	middle_aged	low	yes	excellent	yes
7	8	youth	medium	no	fair	no
8	9	youth	low	yes	fair	yes
9	10	senior	medium	yes	fair	yes
10	11	youth	medium	yes	excellent	yes
11	12	middle_aged	medium	no	excellent	yes
12	13	middle_aged	high	yes	fair	yes
13	14	senior	medium	no	excellent	no

Methods used in Data Encoding

- **LabelEncoder ():** This method is used to normalize labels. It can also be used to transform non-numerical labels to numerical labels.

- **Fit_transform ():** This method is used for Fitting label encoder and return encoded labels.

Data Encoding Procedure

In [4]:
```
import sklearn
from sklearn.preprocessing import LabelEncoder
```

In [5]:
```
le_age = LabelEncoder()
le_income = LabelEncoder()
le_student = LabelEncoder()
le_credit_rating = LabelEncoder()
le_buys_computer = LabelEncoder()
```

In [6]:
```
data['age_n'] = le_age.fit_transform(data['age'])
data['income_n'] = le_income.fit_transform(data['income'])
data['student_n'] = le_student.fit_transform(data['student'])
data['credit_rating_n'] = le_credit_rating.fit_transform(data['credit_rating'])
data['buys_computer_n'] = le_credit_rating.fit_transform(data['buys_computer'])
```

Data Encoding

In [7]:
```
1 data.head()
```

Out[7]:

	RID	age	income	student	credit_rating	buys_computer	age_n	income_n	student_n	credit_rating_n	buys_computer_n
0	1	youth	high	no	fair	no	2	0	0	1	0
1	2	youth	high	no	excellent	no	2	0	0	0	0
2	3	middle_aged	high	no	fair	yes	0	0	0	1	1
3	4	senior	medium	no	fair	yes	1	2	0	1	1
4	5	senior	low	yes	fair	yes	1	1	1	1	1

Structuring Dataframe

drop(): This is used to **Remove** rows or columns by specifying label names and corresponding axis or by specifying directly index or **column** names.

In [8]:
```
1 data_new = data.drop(['age','income','student','credit_rating','buys_computer'], axis='columns')
2 data_new.head()
```

Out[8]:

	RID	age_n	income_n	student_n	credit_rating_n	buys_computer_n
0	1	2	0	0	1	0
1	2	2	0	0	0	0
2	3	0	0	0	1	1
3	4	1	2	0	1	1
4	5	1	1	1	1	1

Independent and Dependent Variables Selection

In [9]:
```
feature_cols = ['age_n', 'income_n', 'student_n', 'credit_rating_n']
x = data_new.drop(['buys_computer_n','RID'], axis='columns') #input
y = data_new['buys_computer_n'] #target
```

In [10]:
```
x.head()
```

Out[10]:

	age_n	income_n	student_n	credit_rating_n
0	2	0	0	1
1	2	0	0	0
2	0	0	0	1
3	1	2	0	1
4	1	1	1	1

In [11]:
```
y.head()
```

Out[11]:
```
0    0
1    0
2    1
3    1
4    1
Name: buys_computer_n, dtype: int32
```

Build the Decision Tree Model without Splitting

In [12]:
```
1 from sklearn.tree import DecisionTreeClassifier
2 clf = DecisionTreeClassifier()
3 dt = clf.fit(x,y)
4 dt
```

Out[12]:
```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
            max_features=None, max_leaf_nodes=None,
            min_impurity_decrease=0.0, min_impurity_split=None,
            min_samples_leaf=1, min_samples_split=2,
            min_weight_fraction_leaf=0.0, presort=False, random_state=None,
            splitter='best')
```

Visualizing Decision Tree

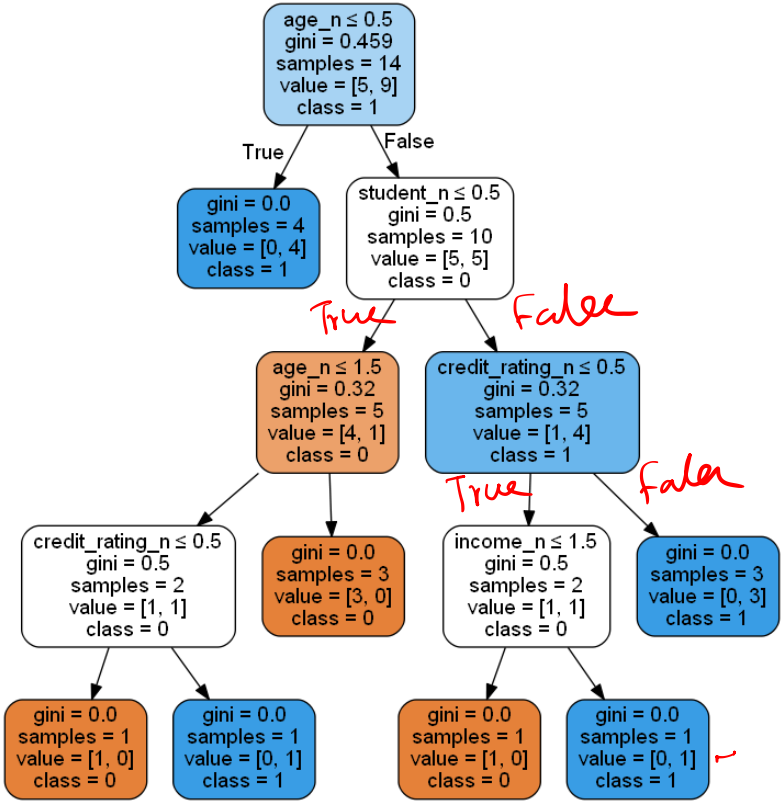
In [13]:
```
from sklearn.tree import export_graphviz
from sklearn.externals.six import StringIO
from IPython.display import Image
import pydotplus
```

In [14]:
```
dot_data = StringIO()
export_graphviz(dt, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True,feature_names = feature_cols,class_names=['0','1'])
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('buys_computer.png')
```

Out[14]: True

In [15]:
```
Image(graph.create_png())
```

Decision Tree Visualization



Interpretation of the CART Output

Calculation of Gini(D)

- We first use the following Equation for Gini index to compute the impurity of D:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2,$$

= $$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

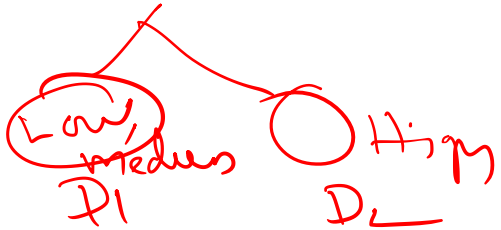
RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Income Attribute

- Low, Medium, High
- Option 1: {Low, Medium}, {High}
- Option 2 : {High, Medium}, {low}
- Option 3 : {High, Low}, {Medium}

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D1



- Low + Medium:

Low + Medium	Class: buys computer
Yes	3+4 =7
No	1+ 2 = 3

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Tuples in partition D2

- High :

High	Class: buys computer
Yes	2
No	2

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

$$Gini_{income \in \{low,medium\}}(D) = \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2)$$

$= (10/14)\,(1 - (7/10)^2 - (3/10)^2) +$

$(4/14)\,(1 - (2/4)^2 - (2/4)^2)$

$= 0.443 = Gini_{\ income \in \{high\}}$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

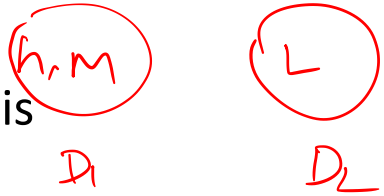
Gini $_{\text{income} \in \{\text{high, medium}\}}$

$$= \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2)$$

$= (10/14)\ (1- (6/10)^2 – (4/10)^2) +$

$(4/14)\ (1- (3/4)^2 – (1/4)^2)$

$=0.45$ = Gini $_{\text{income} \in \{\text{low}\}}$



RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- The Gini index value computed based on this partitioning is

$$\text{Gini}_{\text{income} \in \{\text{high, low}\}} = (8/14)\,(1 - (5/8)^2 - (3/8)^2) + (6/14)\,(1 - (2/6)^2 - (4/6)^2)$$

$$= 0.458 = \text{Gini}_{\text{income} \in \{\text{medium}\}}$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for income attribute

- $Gini_{income \in \{low, medium\}} = 0.443 = Gini_{income \in \{high\}}$
- $Gini_{income \in \{high, medium\}} = 0.45 = Gini_{income \in \{low\}}$
- $Gini_{income \in \{high, low\}} = 0.458 = Gini_{income \in \{medium\}}$

Gini index for Age attribute

- The Gini index value computed based on this partitioning is

$$\text{Gini}_{\text{Age} \in \{\text{Youth, middle\_aged}\}} = 0.457 = \text{Gini}_{\text{Age} \in \{\text{senior}\}}$$

$$\text{Gini}_{\text{Age} \in \{\text{Youth, Senior}\}} = 0.357 = \text{Gini}_{\text{Age} \in \{\text{middle\_aged}\}}$$

$$\text{Gini}_{\text{Age} \in \{\text{senior, middle\_aged}\}} = 0.393 = \text{Gini}_{\text{Age} \in \{\text{Youth}\}}$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for student attribute

- The Gini index value computed based on this partitioning is

$$\text{Gini}_{\text{student} \in \{\text{Yes, No}\}}$$

$$= 7/14\,(1- (6/7)^2 - (1/7)^2\,) +$$

$$7/14\,(1- (3/7)^2 - (4/7)^2\,)$$

$$= 0.367$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for credit_rating attribute

- The Gini index value computed based on this partitioning is

$$\text{Gini}_{\text{credit rating} \in \{\text{fair, Excellent}\}}$$

$$= 8/14\,(1 - (6/8)^2 - (2/8)^2) +$$

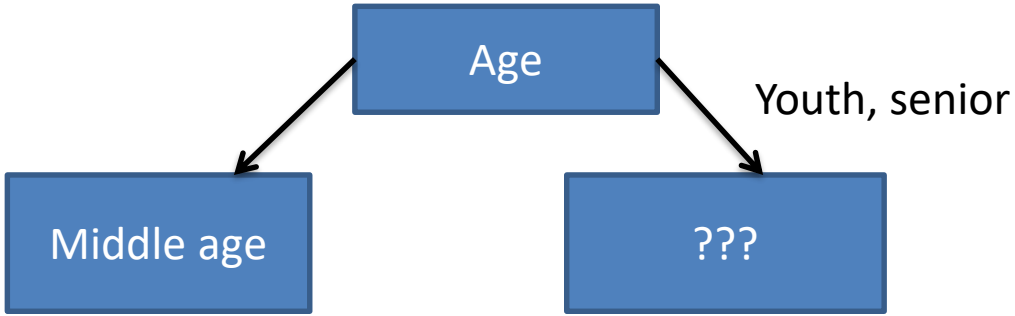
$$6/14\,(1 - (3/6)^2 - (3/6)^2)$$

$$= 0.428$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Choosing the root node

The attribute with minimum Gini score will be taken, i.e. Age (Gini $_{\text{Age} \in \{\text{Youth, Senior}\}}$ = 0.357 = Gini $_{\text{Age} \in \{\text{middle\_aged}\}}$)



Attribute	Gini score
Age	0.357
Income	0.443
Student	0.367
Credit_rating	0.428

Gini index for different attributes for sample of 10

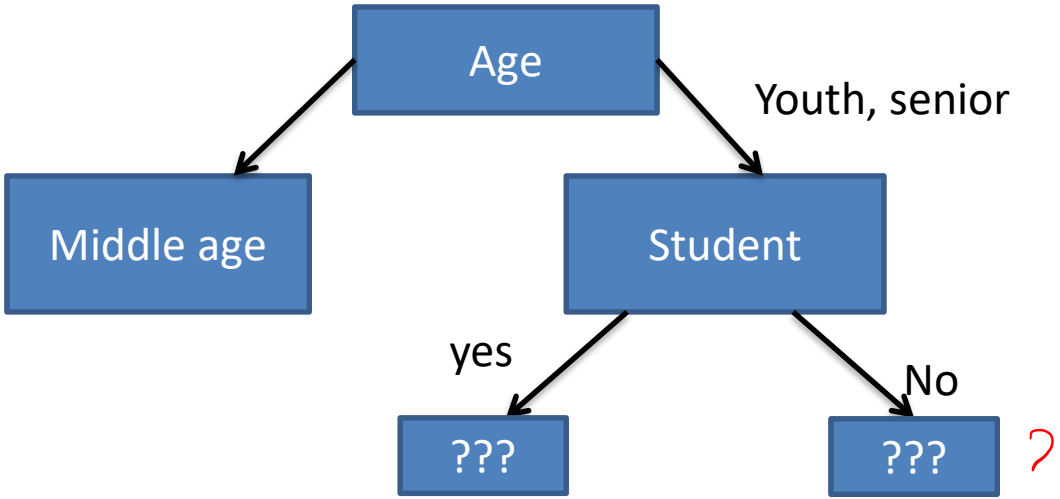
- After separating 4 samples belonging middle age, total 10 are remaining:

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for different attributes for sample of 10

- Gini (D) = $(1- (5/10)^2 – (5/10)^2)) = 0.5$
- $Gini_{Age} = 0.48$
- $Gini_{Credit\ Rating} = 0.41$
- $Gini_{Student} = 0.32$
- $Gini_{income} = 0.375$
- Take student as node as it have mini. Gini Score

Drawing cart



For branch Student = No

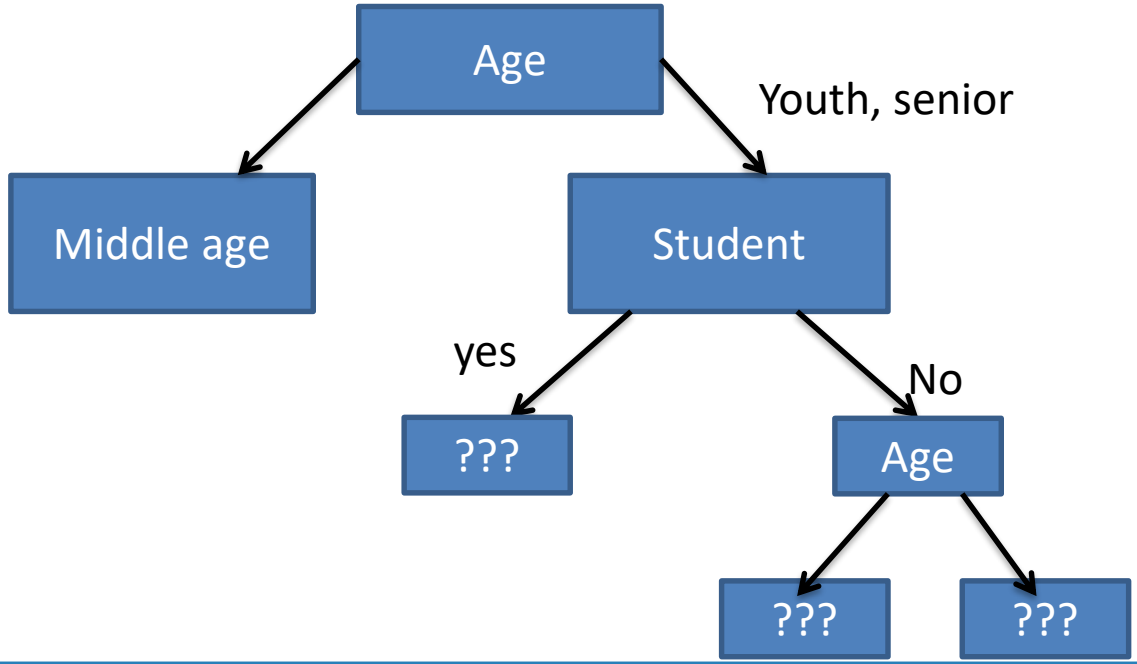
- Omit the marked rows (Data entry), either belonging Age = middle_aged or student = Yes
- Total 5 rows are remaining

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for different attributes For branch Student = No

- Gini (D) = $(1- (4/5)^2 – (1/5)^2)) = 0.32$
- $Gini_{Age} = 0.2$
- $Gini_{Credit\ Rating} = 0.267$
- $Gini_{Student} = 0.32$
- $Gini_{income} = 0.267$
- Take age as node as it have mini. Gini Score

Drawing cart



For branch Student = Yes

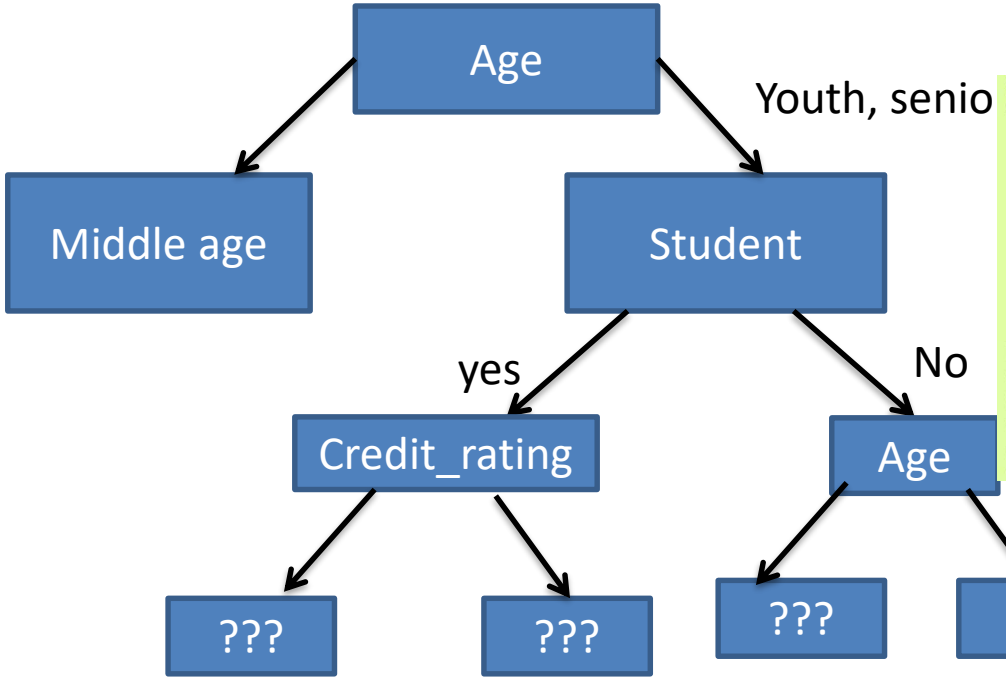
- Omit the marked rows (Data entry), either belonging Age = middle_aged or student = No
- Total 5 rows are remaining

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Gini index for different attributes For branch Student = No

- Gini (D) = $(1- (4/5)^2 – (1/5)^2)) = 0.32$
- $Gini_{Age} = 0.267$
- $Gini_{Credit\ Rating} = 0.2$
- $Gini_{Student} = 0.32$
- $Gini_{income} = 0.267$
- Take credit rating as node as it have mini. Gini Score

Drawing cart



RID	age	income	student	credit_rating	Class: buys_compute
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Coding scheme

Age	Code
Youth	2
Middle Age	0
senior	1

Student	Code
Yes	1
No	0

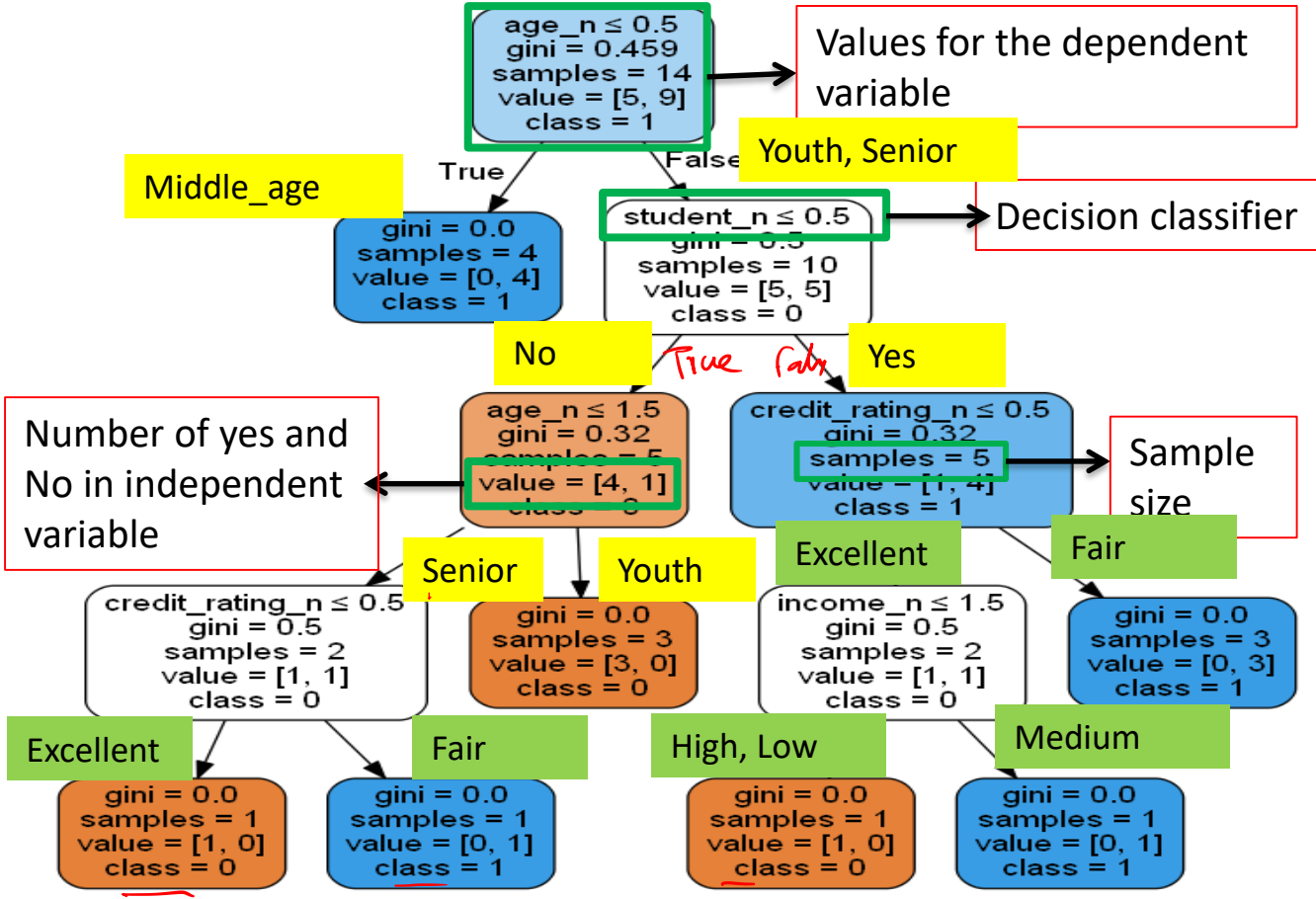
Credit rating	Code
Fair	1
Excellent	0

Income	Code
High	0
Low	1
Medium	2

Buys computer	Class
Yes	1
No	0

Decision tree

- Repeat the splitting process until we obtain all the leaf nodes, the final out - put:



Splitting Dataset

- Train_test_split(): This method is used for splitting dataset into training and testing data subsets.

```
In [12]:  1  from sklearn.model_selection import train_test_split

In [13]:  1  x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=42)
```

Build the Decision Tree Model

In [14]:
```
1 from sklearn.tree import DecisionTreeClassifier
2 clf = DecisionTreeClassifier()
3 dt = clf.fit(x_train,y_train)
4 dt
```

Out[14]:
```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                       max_features=None, max_leaf_nodes=None,
                       min_impurity_decrease=0.0, min_impurity_split=None,
                       min_samples_leaf=1, min_samples_split=2,
                       min_weight_fraction_leaf=0.0, presort=False, random_state=None,
                       splitter='best')
```

Evaluating the Model

```
In [16]:  1  from sklearn import metrics
```

```
In [17]:  1  y_pred = clf.predict(x_test)
```

```
In [18]:  1  print("Accuracy:",metrics.accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.75
```

```
True: [1 1 0 1]
pred: [1 0 0 1]
```

Visualizing Decision Tree

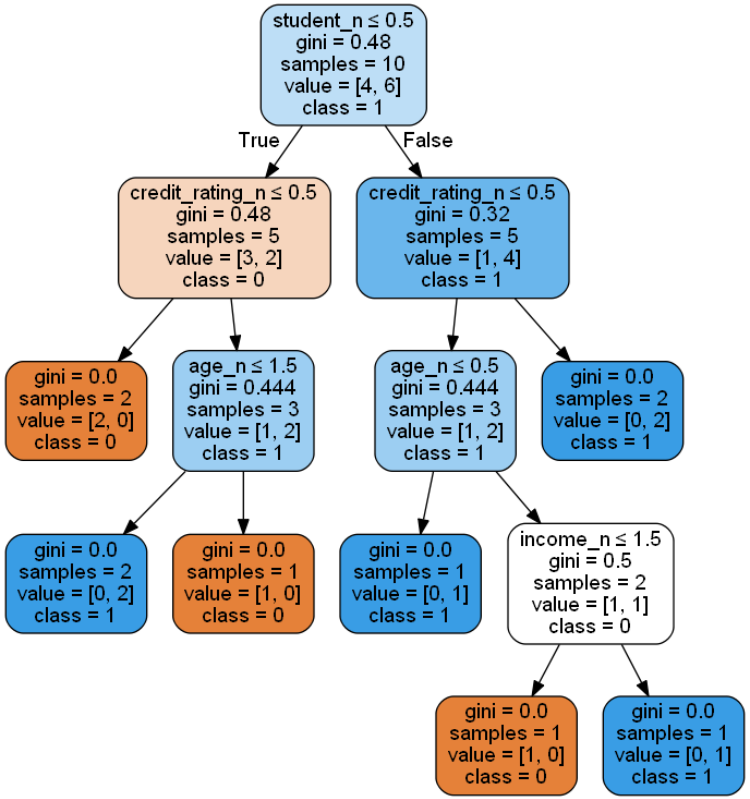
In [19]:
```
from sklearn.tree import export_graphviz
from sklearn.externals.six import StringIO
from IPython.display import Image
import pydotplus
```

In [20]:
```
dot_data = StringIO()
export_graphviz(dt, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True,feature_names = feature_cols,class_names=['0','1'])
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('buys_computer.png')
```

Out[20]: True

In [21]:
```
Image(graph.create_png())
```

Decision Tree Visualization



Thank You